

Министерство образования Российской Федерации

**МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. БАУМАНА**

Факультет: Информатика и системы управления
Кафедра: Информационная безопасность (ИУ8)

**ИНТЕЛЛЕКТУАЛЬНЫЕ ТЕХНОЛОГИИ
ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ**

Лабораторная работа №1 на тему:
**«Исследование однослойных нейронных сетей на примере
моделирования булевых выражений»**

Вариант 13

Преподаватель:
Захаров П.В.

Студент:
Степанова Е.А.

Группа:
ИУ8-21М

Москва 2026

Цель работы

Исследовать функционирование простейшей нейронной сети (НС) на базе нейрона с нелинейной функцией активации и ее обучение по правилу Видроу-Хоффа.

Постановка задачи

Получить модель булевой функции (БФ) на основе однослойной НС (единичный нейрон) с двоичными входами $x_1, x_2, x_3, x_4 \in \{0,1\}$, единичным входом смещения $x_0 = 1$, синаптическими весами w_0, w_1, w_2, w_3, w_4 , двоичным выходом $y \in \{0,1\}$ и заданной нелинейной функцией активации (ФА) $f: R \rightarrow (0,1)$.

Для заданной БФ реализовать обучение НС для двух случаев:

1. с использованием всех комбинаций переменных x_1, x_2, x_3, x_4 ;
2. с использованием части возможных комбинаций переменных x_1, x_2, x_3, x_4 ; остальные комбинации используются в качестве тестовых.

Ход работы

Получим таблицу истинности для моделируемой БФ:

$$(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_2 + \bar{x}_3 + x_4)$$

x_1	x_2	x_3	x_4	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_1 + \bar{x}_2 + \bar{x}_3$	$\bar{x}_2 + \bar{x}_3 + x_4$	F
0	0	0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1
0	0	1	1	1	1	0	1	1	1
0	1	0	0	1	0	1	1	1	1
0	1	0	1	1	0	1	1	1	1
0	1	1	0	1	0	0	1	0	0
0	1	1	1	1	0	0	1	1	1
1	0	0	0	0	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1
1	0	1	0	0	1	0	1	1	1
1	0	1	1	0	1	0	1	1	1
1	1	0	0	0	0	1	1	1	1
1	1	0	1	0	0	1	1	1	1
1	1	1	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	1	0

На начальном шаге $l = 0$ (эпоха $k = 0$) весовые коэффициенты берутся в виде:

$$w_0^{(0)} = w_1^{(0)} = w_2^{(0)} = w_3^{(0)} = w_4^{(0)} = 0$$

Норма обучения для всех случаев выбирается $\eta = 0.3$

1. Обучение НС с использованием всех комбинаций переменных x_1, x_2, x_3, x_4 .

1.1. Используя пороговую ФА:

$$f(net) = \begin{cases} 1, & net > 0, \\ 0, & net \leq 0 \end{cases}$$

Таблица 1 Параметры НС на последовательных эпохах (пороговая ФА)

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	Y = (1,1,1,1,1,1,1,0,1,1,1,1,1,1,0), W = (-0.30, -0.30, -0.30, -0.30, 0.30), E = 3
1	Y = (0,1,0,1,1,1,1,0,1,1,1,1,0,1,1), W = (0.0, -0.60, -0.60, -0.60, 0.30), E = 7
2	Y = (1,1,0,1,0,1,1,0,1,1,0,1,1,1,0), W = (0.00, -0.60, -0.60, -0.30, 0.60), E = 6
3	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0), W = (0.6, -0.6, -0.9, -0.3, 0.6), E = 2
4	Y = (1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1), W = (0.6, -0.9, -0.9, -0.9, 0.6), E = 6
5	Y = (1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1), W = (1.2, -0.6, -0.9, -0.6, 0.6), E = 4
6	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1), W = (1.2, -0.6, -0.9, -0.9, 0.6), E = 2
7	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0), W = (1.2, -0.6, -1.2, -0.9, 0.3), E = 2
8	Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0), W = (1.20, -0.90, -1.20, -0.90, 0.30), E = 2

9	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0)$ $W = (1.5, -0.9, -0.9, -0.9, 0.6), E = 3$
...	...
28	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0)$ $W = (2.7, -0.9, -1.8, -1.2, 0.9), E = 0$

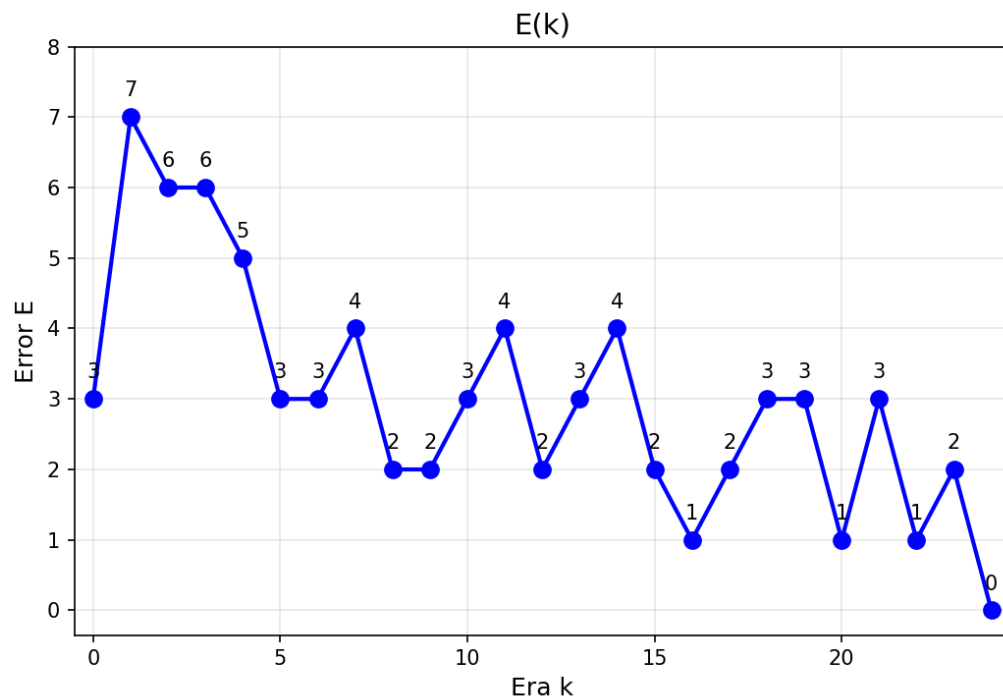


Рисунок 1 График суммарной ошибки НС по эпохам обучения (пороговая ФА)

1.2. Используя сигмоидальную (логистическую) ФА:

$$f(net) = \frac{1}{2} \left(\frac{net}{1 + |net|} + 1 \right)$$

производная, которой

$$\frac{df(net)}{dnet} = \frac{1}{2} * \frac{1}{(1 + |net|)^2}$$

Таблица 2 Параметры НС на последовательных эпохах (логистическая ФА)

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	Y = (1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0), W = (-0.0576, -0.0656, -0.0576, -0.0576, 0.0747), E = 3
1	Y = (0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1), W = (0.0303, -0.1208, -0.1076, -0.107, 0.082), E = 7
2	Y = (1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1), W = (0.1099, -0.1826, -0.098, -0.0944, 0.0877), E = 5
3	Y = (1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0), W = (0.184, -0.1085, -0.1616, -0.0901, 0.0877), E = 3
4	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0), W = (0.1857, -0.1068, -0.2272, -0.0884, 0.0877), E = 2
...	...
9	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0), W = (0.3525, -0.1515, -0.1987, -0.1919, 0.1677), E = 0

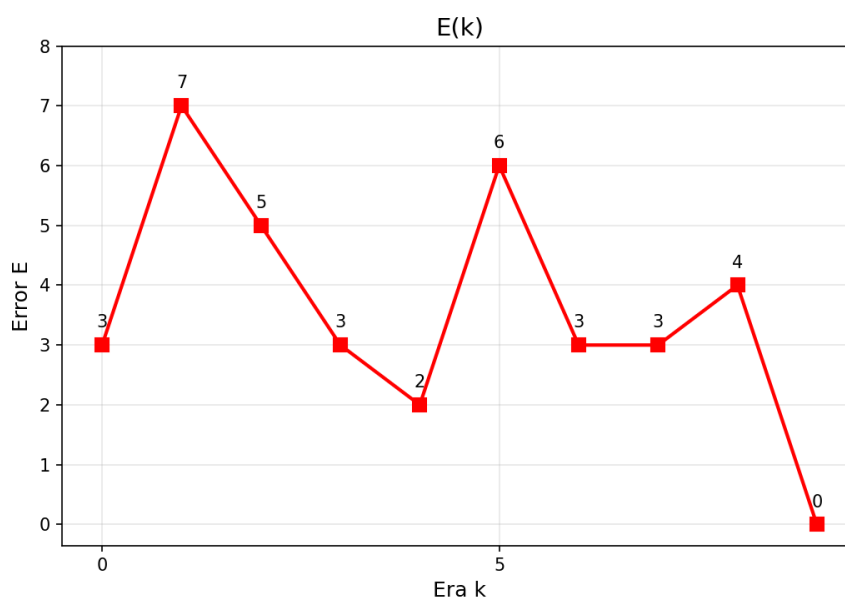


Рисунок 2 График суммарной ошибки НС по эпохам обучения (логистическая ФА)

2. Обучение НС с использованием части комбинаций переменных x_1, x_2, x_3, x_4

Последовательно уменьшая выборку количества векторов, найдём наименьшее количество необходимых для обучения векторов.

2.1. Используя пороговую ФА.

Минимальный набор из четырёх векторов:

$$X^{(1)} = (0, 0, 1, 0), X^{(2)} = (1, 0, 0, 1), X^{(3)} = (1, 1, 0, 0), X^{(4)} = (1, 1, 1, 0)$$

Даёт следующие синаптические коэффициенты:

$$W = (1.2, -0.3, -0.6, -0.9, 0.3)$$

Для полного обучения потребовалось 8 эпох.

Таблица 3 Параметры НС на последовательных эпохах (пороговая ФА) при наборе из 4 векторов

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	Y = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), W = (-0.300, -0.300, -0.300, -0.300, 0.000), E = 13
1	Y = (1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0), W = (0.000, -0.300, -0.600, -0.300, 0.300), E = 9
2	Y = (1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0), W = (0.300, -0.300, -0.600, -0.300, 0.300), E = 5
...	...
8	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0), W = (1.200, -0.300, -0.600, -0.900, 0.300), E = 0

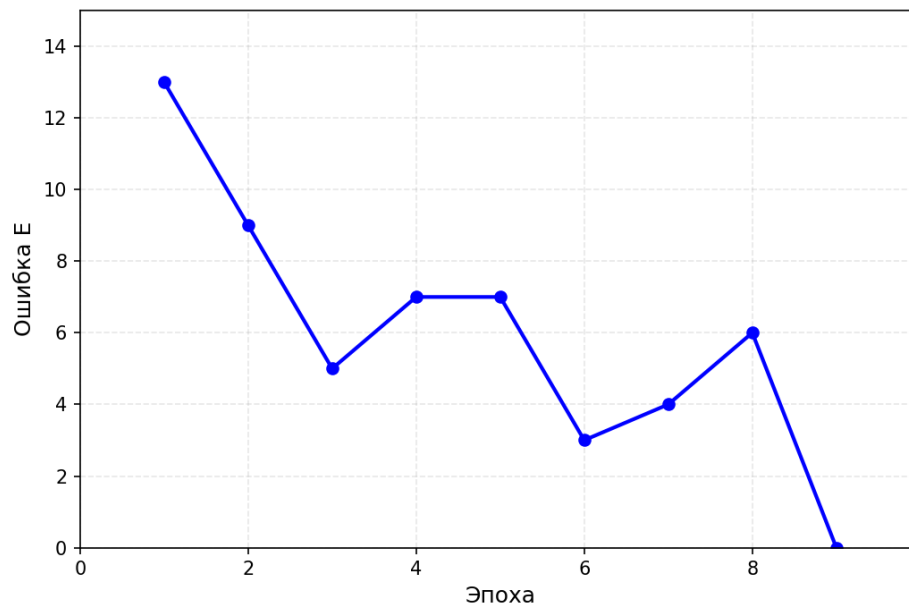


Рисунок 3 График суммарной ошибки НС по эпохам обучения (пороговая ФА) при наборе из 4-х векторов

2.2. Используя сигмоидальную (логистическую) ФА.
Минимальный набор из четырёх векторов:

$$X^{(1)} = (0, 1, 0, 0), X^{(2)} = (1, 0, 1, 0), X^{(3)} = (1, 1, 0, 1), X^{(4)} = (1, 1, 1, 0)$$

Даёт следующие синаптические коэффициенты:

$$W = (0.747, -0.052, -0.160, -0.123, 0.071)$$

Для полного обучения потребовалось 19 эпох.

Таблица 4 Параметры НС на последовательных эпохах (логистическая ФА) при наборе из 4 векторов

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	Y = (0,0,0,0,0,0,0,0,0,0,0,0,0,0,0), W = (-0.208, 0.058, 0.094, 0.058, 0.000), E = 13
1	Y = (0,0,0,0,0,0,1,1,0,0,1,1,1,1,1) W = (0.326, 0.088, 0.137, 0.088, 0.000), E = 11
2	Y = (0,0,0,0,1,1,1,1,0,0,0,0,0,1,1,1,1), W = (0.349, 0.040, 0.160, 0.040, 0.00), E = 11

3	$Y = (0,0,0,0,0,0,1,1,0,0,1,1,1,1,1,1),$ $W = (0.376, 0.068, 0.114, 0.068, 0.000), E = 0$
...	...
19	$Y = (1,1,1,1,1,1,0,1,1,1,1,1,1,1,0,0),$ $W = (0.747, -0.052, -0.160, -0.123, 0.071), E = 0$

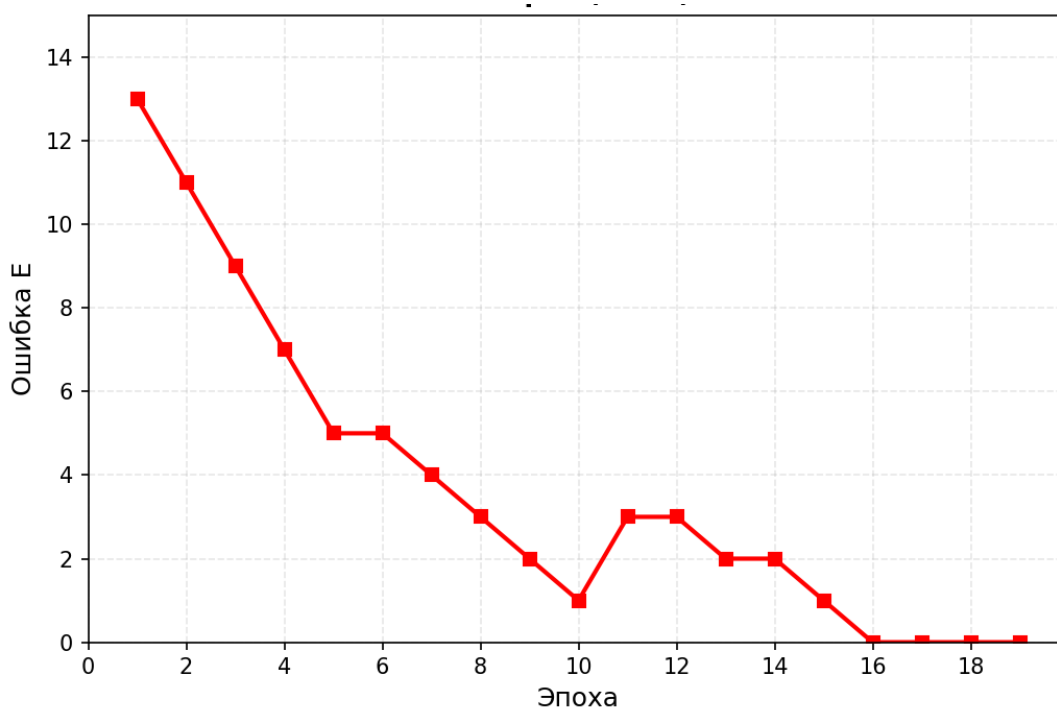


Рисунок 4 График суммарной ошибки НС по эпохам обучения (логистическая ФА) при наборе из 4-х векторов

Выводы

В ходе проделанной работы была изучена работа простейшего нейрона НС на базе нейрона с нелинейной функцией активации и проведено обучение по правилу Видроу-Хоффа. В качестве функций активации брались две различные функции – пороговая и логистическая. В ходе обучения на полных наборах было выявлено, что с использованием логистической функции активации понадобилось меньше эпох, чем для обучения с использованием пороговой функции активации.

Кроме того, для случаев пороговой и логистической функций активации были найдены минимально возможные наборы векторов, на которых можно

обучить НС. В обоих случаях удалось найти наборы, состоящие из четырёх векторов. В случае обучения с использованием пороговой функции активации понадобилось меньшее количество эпох, чем с использованием логистической.

Приложение А.

Файл 'lab-1.py'.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import combinations

def boolean_function(x1, x2, x3, x4):
    return 1 if (not (x2 and x3)) or ((not x1) and x4) else 0

n = 4
X, targets = [], []

for i in range(16):
    x1 = (i >> 3) & 1
    x2 = (i >> 2) & 1
    x3 = (i >> 1) & 1
    x4 = (i >> 0) & 1

    t = boolean_function(x1, x2, x3, x4)
    X.append([x1, x2, x3, x4])
    targets.append(t)

    x_str = f"{x1} {x2} {x3} {x4}"
    print(f"{i+1:2} | {x_str} | {t}")

def activate(net, type='1'):
    if type == '1':
        return 1.0 if net >= 0 else 0.0
    else:
        return 1.0 if net >= 0.5 else 0.0

def sigma(net):
    return 0.5 * (net / (1 + abs(net)) + 1)

def dsigma(net):
    return 1.0 / (2.0 * (1 + abs(net)) ** 2)
```

```

def train_neuron(type):
    w = np.zeros(5)
    error_history = np.zeros(50)
    y_net = np.zeros(16)
    e = 0
    delta = np.zeros(5)

    for ephoch in range(29):

        print("\nЭпоха", ephoch)
        print("\n")
        for i in range(16):
            net = round((w[0] + w[1]*X[i][0] + w[2]*X[i][1] + w[3]*X[i][2] + w[4]*X[i][3]), 3)
            if type == '2':
                net = sigma(net)
            y = activate(net, type)
            y_net[i] = y
            t = targets[i]
            error = t - y
            if error != 0:
                error_history[e] = error_history[e] + 1

                for j in range(5):

                    if j == 0:
                        if type == '1':
                            delta[0] = delta[0] + 1 * 0.3 * error
                        else:
                            delta[0] = round((delta[0] + 1 * 0.3 * error * dsigma(net)), 4)
                    else:
                        if type == '1':
                            delta[j] = delta[j] + 0.3 * error * X[i][j-1]
                        else:
                            delta[j] = round((delta[j] + 0.3 * error * X[i][j-1] *
dsigma(net)), 4)

                w = delta
                print("Суммарная ошибка E = ", error_history[e])
                e = e + 1
                print("Вектор весов W = ", delta)
                print("Выходной вектор = ", y_net)

        return error_history

def test_full_set(w, X, targets, type):

    for i in range(len(X)):
        net = w[0] + w[1]*X[i][0] + w[2]*X[i][1] + w[3]*X[i][2] + w[4]*X[i][3]
        if type == '2':
            net = sigma(net)
        y = activate(net, type)
        if y != targets[i]:
            return False
    return True

```

```

def train_on_subset(subset_X, subset_targets, X, targets, type, max_epochs=50, verbose=True):

    w = np.zeros(5)
    epoch_errors = []
    epoch_outputs = []

    for epoch in range(max_epochs):
        epoch_error = 0
        epoch_output = []

        # Обучение на подмножестве
        for i in range(len(subset_X)):
            net = w[0] + w[1]*subset_X[i][0] + w[2]*subset_X[i][1] + w[3]*subset_X[i][2] +
w[4]*subset_X[i][3]
            y = activate(net, type)
            error = subset_targets[i] - y

            if error != 0:
                if type == '1':
                    w[0] += 0.3 * error
                else:
                    w[0] += 0.3 * error*dsigma(net)

                for j in range(1, 5):
                    if type == '1':
                        w[j] += 0.3 * error * subset_X[i][j-1]
                    else:
                        w[j] += 0.3 * error * subset_X[i][j-1]*dsigma(net)

        all_outputs = []
        for i in range(len(X)):

            net = w[0] + w[1]*X[i][0] + w[2]*X[i][1] + w[3]*X[i][2] + w[4]*X[i][3]
            y = activate(net, type)
            all_outputs.append(y)
            if y != targets[i]:
                epoch_error += 1

        epoch_errors.append(epoch_error)
        epoch_outputs.append(all_outputs)

        if epoch_error == 0:
            print(f"Обучение завершено на эпохе {epoch}")
            return True, w, epoch + 1, epoch_errors, epoch_outputs

    return False, None, max_epochs, epoch_errors, epoch_outputs

def find_minimal_set(X, targets, type):
    indices = list(range(len(X)))

    # Идем от размера 1 до 16, перебирая все комбинации

```

```

for size in range(1, len(X) + 1):

    for combo in combinations(indices, size):
        subset_X = [X[i] for i in combo]
        subset_targets = [targets[i] for i in combo]

        # Пытаемся обучиться на текущей комбинации
        success, w_final, epochs, epoch_errors, epoch_outputs = train_on_subset(
            subset_X, subset_targets, X, targets, type, max_epochs=50, verbose=True
        )

        if success:
            print(f"Размер набора: {size}")
            print(f"Индексы векторов (0-15): {combo}")
            print("\nВекторы в обучающем наборе:")
            print(f"{'i':>3} | {'x1 x2 x3 x4':^12} | t")
            print("-" * 30)
            for i in combo:
                print(f"{i:3} | {X[i][0]} {X[i][1]} {X[i][2]} {X[i][3]} | {targets[i]}")
            print(f"Итоговые веса [w0, w1, w2, w3, w4]: {np.round(w_final, 4)}")

            return combo, w_final, epoch_errors, epoch_outputs

    return None, None, None, None

# Запускаем алгоритм поиска минимального набора
train_neuron('1')
train_neuron('2')

find_minimal_set(X, targets, '1')

find_minimal_set(X, targets, '2')

```